

#### Esercizio 4

a) Descrivere brevemente l'algoritmo MFQ con Affinity Scheduling per sistemi multiprocessore.

b) Una struttura dati è condivisa da 4 thread T1, T2, T3, T4. I thread T1 e T2 modificano la struttura condivisa, i thread T3 e T4 accedono alla struttura in sola lettura. Per l'accesso alla struttura dati si adotta la seguente politica:

- Inizialmente accede il primo thread che ne fa richiesta sia esso lettore o scrittore;
- Quando la struttura dati è in uso da T3 o da T4 (o da entrambi), se i thread T1 e T2 decidono di accedere alla struttura dati vengono messi in attesa. I thread in attesa vengono riattivati solo quando sia T3 sia T4 hanno completato l'accesso alla struttura dati. Se T1 e T2 sono entrambi in attesa allora viene riattivato uno dei due. Se né T1 né T2 sono in attesa allora la struttura dati ritorna disponibile a tutti i thread e si torna alla situazione iniziale;
- Quando la struttura dati condivisa è utilizzata da T1 (o da T2), tutti gli altri thread che richiedono l'accesso vengono messi in attesa. Quando T1 (oppure T2) termina il proprio accesso, verifica prima se T2 (o T1) è in attesa e in caso affermativo lo riattiva. Altrimenti, se sono in attesa, vengono riattivati T3 e/o T4. Se nessun thread è in attesa allora la struttura dati ritorna disponibile a tutti i thread e si torna alla situazione iniziale.

Si chiede di implementare, utilizzando esclusivamente i semafori come meccanismo di sincronizzazione, i metodi: `accediT1T2()`, `rilasciaT1T2()`, `accediT3T4()`, `rilasciaT3T4()`.

#### • Soluzione

##### ↳ Passo 1: Identificare il problema

Abbiamo 4 Thread che accedono ad una struttura dati condivisa con regole specifiche

- $T_1$  e  $T_2$  (scrittori): solo uno alla volta può accedere
- $T_3$  e  $T_4$  (lettori): possono accedere contemporaneamente, ma solo se nessuno scrittore è attivo / in attesa

⇒ Bisogna garantire mutua esclusione tra lettori e scrittori, con una gestione ordinata delle attese

##### ↳ Passo 2: Identificare le variabili necessarie

Bisogna tenere traccia dello stato della struttura dati e gestire l'accesso con semafori e contatori:

#### • Semafori

→ **mutex**: controlla l'accesso in mutua esclusione alle variabili condivise

→ **waitT1T2**: gestisce l'attesa degli scrittori

→ **waitT3T4**: gestisce l'attesa dei lettori.

- Contatori

→  $T1T2$ : indica se uno scrittore è nella sezione critica ( $\emptyset$  o 1)

→  $T1T2_{waiting}$ : conta gli scrittori in attesa

→  $T3T4$ : conta i lettori attivi nella sezione critica

→  $T3T4_{waiting}$ : conta i lettori in attesa

⇒ Obiettivo: Usare questi semafori e contatori per rispettare la politica di accesso richiesta.

### ↳ Passo 3: Definire il comportamento degli scrittori ( $T1$ e $T2$ )

Gli scrittori devono rispettare le seguenti condizioni:

1. Se nessun lettore / scrittore è attivo → entra subito
2. Se ci sono lettori attivi o un altro scrittore → va in attesa su  $wait(T1T2)$
3. Quando uno scrittore termina, se ce ne sono altri in attesa, ne attiva 1 solo
4. Se nessuno scrittore è in attesa, attiva tutti i lettori in attesa

- Passaggi per  $accedi(T1T2C)$ :

→ Entrare in mutua esclusione ( $PC_{mux}$ )

→ Controllare se la sezione critica è libera ( $T3T4 == 0, T1T2 == 0$ )

- Se sì → lo scrittore entra ( $T1T2 == 1$ )

- Se no → incrementa  $T1T2_{waiting}$  e si mette in attesa ( $PC_{wait(T1T2)}$ )

→ Uscire dalla mutua esclusione ( $VC_{mux}$ )

- Passaggi per  $rilascia(T1T2C)$

→ Entrare in mutua esclusione ( $PC_{mux}$ )

→ Segnalare che lo scrittore ha finito ( $T1T2 == 0$ )

→ Se ci sono scrittori in attesa → attivarne uno ( $VC_{wait(T1T2)}$ )

→ Se non ci sono scrittori in attesa → attivare tutti i lettori ( $VC_{wait(T3T4)}$ )

→ Uscire dalla mutua esclusione ( $VC_{mux}$ )

## ↳ Passo 4: Definire il comportamento dei lettori

I lettori devono rispettare queste condizioni:

1. Se non ci sono scrittori attivi/in attesa  $\rightarrow$  entra subito.
2. Se c'è uno scrittore attivo/in attesa  $\rightarrow$  devono aspettare ( $\text{wait } T3T4$ )
3. Quando un lettore esce, se è l'ultimo lettore e ci sono scrittori in attesa  $\rightarrow$  deve attivare uno scrittore

### • Passaggi per $\text{accedi}(T3T4)$

$\rightarrow$  Entrare in mutua esclusione ( $P(\text{mux})$ )

$\rightarrow$  Controllare se ci sono scrittori in attesa ( $T1T2\text{waiting} > 0$ )

o attivi ( $T1T2 == 1$ )

- Se no  $\rightarrow$  può entrare subito ( $T3T4++$ )

- Se si  $\rightarrow$  incrementa  $T3T4\text{ waiting}$  e si mette in attesa ( $P(\text{wait } T3T4)$ )

$\rightarrow$  Uscire dalla mutua esclusione ( $V(\text{mux})$ )

### • Passaggi per $\text{rilascia}(T3T4)$

$\rightarrow$  Entrare in mutua esclusione ( $P(\text{mux})$ )

$\rightarrow$  Decrementare  $T3T4$

$\rightarrow$  Se è l'ultimo lettore e ci sono scrittori in attesa, attivare uno scrittore ( $V(\text{wait } T1T2)$ )

$\rightarrow$  Uscire dalla mutua esclusione ( $V(\text{mux})$ )

## • Dalla logica allo PSEUDOCODICE

### ↳ Passo 1: Dichiarazioni delle variabili locali

Definisco le variabili globali che servono per la sincronizzazione

sem\_mux = 1; // controlla l'accesso alle variabili condivise

sem\_waitT1T2 = 0; // Attesa degli scrittori

sem\_waitT3T4 = 0; // Attesa dei lettori

// Contatori

T1T2 = 0; // Indica se c'è uno scrittore attivo (0 o 1)

T1T2\_waiting = 0 // # scrittori in attesa

T3T4 = 0; // # lettori attivi

T3T4\_waiting = 0; // # lettori in attesa

### ⚠ Achtung:

- "mux" viene usato come **mutua esclusione** per modificare i contatori
- "waitT1T2" e "waitT3T4" sono usati per **mettere in attesa i Thread**
- I contatori ci permettono di **tenere traccia dello stato** della sezione critica

### ↳ Passo 2: Implementazione di `accedi(T1T2C)`

Questa funzione deve:

- Verificare se ci sono lettori o scrittori attivi
- Se la sezione critica è libera, entra immediatamente
- Altrimenti, si mette in attesa su "waitT1T2"

```
void accedi(T1T2C) {
```

```
    bool ok = false; // Flag per sapere se entrare subito
```

```
    PC & mux); // Entra in mutua esclusione
```

```
    if (T3T4 == 0 && T1T2 == 0) { // Nessun lettore / scrittore attivo
```

```
        T1T2 = 1;
```

```
        ok = true; // Può entrare immediatamente
```

```
    } else {
```

```
        T1T2 waiting ++; // Incrementa # scrittori in attesa
```

```
    {
```

```
        VC & mux); // Esce dalla mutua esclusione
```

```
    if (!ok) {
```

```
        PC & wait(T1T2); // Se non può entrare subito → va in wait
```

```
    }
```

```
    }
```

### ! Achtung

- PC & mux) / VC & mux) proteggono le variabili condivise
- Se la sezione critica è libera (T1T2 == 0 e T3T4 == 0), lo scrittore entra subito.
- Altrimenti, aumenta il contatore degli scrittori in attesa e aspetta su wait(T1T2).

### ↳ Passo 3: Implementazione di `rilasciaT1T2C()`

Questa funzione deve:

- Segnalare che lo scrittore ha finito.
- Se ci sono scrittori in attesa, svegliarne uno solo.
- Se non ci sono, svegliare tutti i lettori.

```
void rilasciaT1T2C()
```

```
PC & mux); // Entra in mutua esclusione
```

```
T1T2 = 0; // Lo scrittore esce dalla sezione critica
```

```
if (T1T2waiting > 0) { // Se ci sono scrittori in attesa
```

```
    T1T2 = 1; // Un nuovo scrittore entra
```

```
    T1T2waiting --; // Riduce #scrittori in attesa
```

```
    VC & waitT1T2); // Risveglia uno scrittore
```

```
    } else {
```

```
        while (T3T4waiting > 0) { // Se non ci sono scrittori, sveglia i lettori
```

```
            T3T4waiting --;
```

```
            VC & waitT3T4);
```

```
        }
```

```
    }
```

```
    VC & mux); // Esce dalla mutua esclusione
```

#### ⚠ Achtung:

- Se ci sono scrittori in attesa, ne sveglia uno solo
- Se non ci sono scrittori, risveglia tutti i lettori in attesa
- Il ciclo "`while (T3T4waiting > 0)`" assicura che tutti i lettori vengano risvegliati.

## L3 Passo 4: Implementazione di `accedi(T3T4C)`

Questa funzione deve:

- Controllare se un lettore può entrare subito o se deve aspettare
- Se non ci sono scrittori attivi o in attesa, entra immediatamente
- Altrimenti, si mette in attesa su `waitT3T4`

```
void accedi(T3T4C) {
```

```
    bool ok = false; // Flag per sapere se entrare subito
```

```
    PC & mux); // Entra in mutua esclusione
```

```
    if (T1T2 == 0 && T1T2 == 0) { // Nessuno scrittore attivo o in attesa
```

```
        T3T4++; // Il lettore entra nella sezione critica
```

```
        ok = true; // Può entrare immediatamente
```

```
    } else {
```

```
        T3T4waiting++; // Incrementa #lettori in attesa
```

```
    {
```

```
        VC & mux); // Esce dalla mutua esclusione
```

```
        if (!ok) {
```

```
            PC & waitT1T2); // Se non può entrare subito -> va in wait
```

```
        {
```

```
    } }
```

### ⚠ Achtung

- Se non ci sono scrittori attivi o in attesa, il lettore può entrare subito
- Se ci sono scrittori, il lettore si mette in attesa (`T3T4waiting++`)

### ↳ Passo 5: Implementazione di `rilasciaT3T4C()`;

Questa funzione deve:

→ Segnalare che un lettore ha finito.

→ Se è l'ultimo lettore, attivare uno scrittore in attesa.

`void rilasciaT3T4C()` {

`PC&mux); // Entra in mutua esclusione`

`T3T4--; // Il lettore esce dalla sezione critica`

`if (CT3T4 == 0 && T1T2waiting > 0) { // Se era l'ultimo lettore e ci sono scrittori in attesa`

`T1T2 = 1; // Uno scrittore può entrare`

`T1T2waiting--; // Riduce # scrittori in attesa`

`VC&waitT1T2); // Risveglia uno scrittore`

`{`

`VC&mux); // Esce dalla mutua esclusione`

`}`

### ⚠ Achtung

- Solo l'ultimo lettore può svegliare uno scrittore in attesa
- Il controllo "`if (CT3T4 == 0 && T1T2waiting > 0)`" assicura che non si verifichino accessi simultanei.